

GRENOBLE-INP ENSIMAG

INGÉNIERIE DES SYSTÈMES
D'INFORMATION

2A ISI

GRENOBLE-INP ENSIMAG

INFORMATION SYSTEMS
ENGINEERING

2A ISI



**ÉCOLE NATIONALE SUPÉRIEURE
D'INFORMATIQUE ET DE MATHÉMATIQUES
APPLIQUÉES DE GRENOBLE**

PROJET GoMile

Documentation technique du plugin WooCommerce

Réalisé par :

DUONG DANG KHOA DANG
DIAKITE ALPHA OUSMANE
JOUNI YOUSSEF
MAOUDE SAMIR
OUDRHIRI IDRISSE SAFWANE

CLIENT :

GoMile Entreprise

2026–2027

Table des matières

1	Objectif du document	3
1.1	Structure du plugin	3
2	Installation et exécution	4
2.1	Prérequis locaux	4
2.2	Installer le plugin dans WordPress	4
2.3	Exécuter les tests	4
3	Notions WordPress essentielles	5
3.1	Plugin WordPress	5
3.2	<code>add_action</code>	5
3.3	<code>add_filter</code>	6
3.4	Définition de points d'extension	7
3.5	Fonctions de rappel	7
3.6	Options WordPress	8
3.7	API de réglages WordPress	8
3.8	Capacités	9
3.9	Nettoyage et échappement des données	9
3.10	Internationalisation	9
3.11	API HTTP WordPress	12
3.12	<code>WP_Error</code>	13
3.13	Transients	13
3.14	API REST WordPress	13
4	Notions WooCommerce essentielles	14
4.1	Méthode d'expédition	14
4.2	Initialisation des méthodes d'expédition	14
4.3	Zones et instances	14
4.4	<code>calculate_shipping</code>	15
4.5	Commandes WooCommerce	15
4.6	Métadonnées de commande	16
4.7	Hooks de validation de commande	16

4.8	Statuts de commande	17
5	Architecture du plugin Gomile	17
5.1	Séquence de chargement	17
5.2	Configuration	18
5.3	Flux de devis	19
5.4	Flux de création de livraison	21
5.5	Flux d'annulation	23
5.6	Flux de webhook	23
6	API Gomile utilisée	25
7	Points d'extension du plugin	25
7.1	Modifier les en-têtes API	25
7.2	Modifier les arguments HTTP	25
7.3	Modifier le tarif WooCommerce final	26
8	Sécurité et robustesse	26
8.1	Secrets	26
8.2	Webhooks	26
8.3	Idempotence	26
9	Tests	27
10	Glossaire	27
11	Références officielles	28

Gomile

1 Objectif du document

Ce document présente l'architecture technique du plugin **Gomile Shipment**. Il décrit les mécanismes WordPress et WooCommerce mobilisés par le plugin, puis les relie aux fichiers et aux flux implémentés dans le projet.

Le plugin ajoute une méthode de livraison WooCommerce connectée à l'API Gomile. Ses fonctionnalités principales sont les suivantes :

- afficher une méthode d'expédition **Gomile Delivery** au panier et au checkout ;
- demander un devis de livraison à l'API Gomile ;
- créer une commande de livraison Gomile après une commande WooCommerce ;
- annuler la livraison Gomile si la commande WooCommerce est annulée ;
- recevoir des webhooks Gomile pour mettre à jour les statuts de livraison.

1.1 Structure du plugin

Fichier	Rôle
<code>gomile-shipment.php</code>	Point d'entrée du plugin, chargement des fichiers, traductions et vérification de WooCommerce.
<code>includes/class-admin-settings.php</code>	Réglages d'administration, valeurs par défaut, nettoyage des données et accès à la configuration.
<code>includes/class-shipping-method.php</code>	Déclaration de la méthode d'expédition WooCommerce et calcul du tarif.
<code>includes/class-delivery-api.php</code>	Client HTTP vers l'API Gomile, construction des charges JSON, cache des devis et lecture des réponses.
<code>includes/class-order-handler.php</code>	Hooks de commande : création et annulation des livraisons Gomile.
<code>includes/class-rest-endpoints.php</code>	Point d'accès REST de webhook et mise à jour des commandes depuis Gomile.
<code>README.md</code>	Instructions d'installation, de configuration et de diagnostic.

Fichier	Rôle
tests/Unit/*.php	Tests unitaires décrivant les comportements attendus.

2 Installation et exécution

2.1 Prérequis locaux

- WordPress installé.
- WooCommerce installé et actif.
- PHP compatible avec la version WordPress/WooCommerce utilisée.
- Composer pour les tests unitaires du plugin.
- Accès à une API Gomile ou à un environnement de test.

2.2 Installer le plugin dans WordPress

Le dossier source est :

```
apps/plugin/woocommerce
```

Dans une installation WordPress, le plugin doit être placé comme suit :

```
wp-content/plugins/gomile-shipment/gomile-shipment.php
```

Le nom du dossier est important pour avoir une URL et un chargement cohérents.

2.3 Exécuter les tests

Depuis le dossier du plugin :

```
cd apps/plugin/woocommerce
composer install
composer test
```

Commandes complémentaires :

```
composer lint
composer test:unit
composer docs
```

3 Notions WordPress essentielles

3.1 Plugin WordPress

Un plugin WordPress est un dossier placé dans `wp-content/plugins`. Son fichier principal contient un commentaire d'en-tête lu par WordPress :

```
/**
 * Plugin Name: Gomile Shipment
 * Description: WooCommerce shipping method to assign a delivery driver via
 *             Gomile.
 * Version: 1.1.0
 * Text Domain: gomile-shipment
 */
```

Dans ce projet, le fichier principal est `gomile-shipment.php`. Il définit des constantes de version et de chemin :

- `GOMILE_SHIPMENT_VERSION` : version du plugin ;
- `GOMILE_SHIPMENT_PLUGIN_FILE` : chemin du fichier principal ;
- `GOMILE_SHIPMENT_PLUGIN_DIR` : dossier local du plugin ;
- `GOMILE_SHIPMENT_PLUGIN_URL` : URL publique du dossier.

Presque tous les fichiers commencent par :

```
if (!defined('ABSPATH')) {
    exit;
}
```

`ABSPATH` est défini par WordPress. Cette garde empêche l'exécution directe d'un fichier PHP du plugin depuis le navigateur.

3.2 `add_action`

`add_action` accroche une fonction à un événement WordPress ou WooCommerce. Quand l'événement est déclenché par WordPress, la fonction enregistrée est appelée.

Signature simplifiée :

```
add_action($hook_name, $callback, $priority = 10, $accepted_args = 1);
```

- `$hook_name` : nom de l'événement.
- `$callback` : fonction, méthode statique ou méthode d'objet.

- `$priority` : ordre d'exécution. Plus le nombre est petit, plus le callback part tôt.
- `$accepted_args` : nombre d'arguments transmis au callback.

Exemple du plugin :

```
add_action('plugins_loaded', 'gomile_shipment_init_plugin', 20);
```

Ici, WordPress déclenche `plugins_loaded` une fois les plugins chargés. Le plugin initialise ses réglages, charge les classes et vérifie que WooCommerce est actif.

Autre exemple :

```
add_action(
    'woocommerce_checkout_order_processed',
    'gomile_shipment_handle_order',
    10,
    1
);
```

Quand une commande est créée par le checkout classique, WooCommerce appelle `gomile_shipment_handle_order($order_id)`.

3.3 add_filter

`add_filter` accroche une fonction à un filtre. Un filtre reçoit une valeur, peut la modifier, puis doit la retourner.

Signature simplifiée :

```
add_filter($hook_name, $callback, $priority = 10, $accepted_args = 1);
```

Différence entre action et filtre :

- une action sert à **déclencher un traitement** à un moment donné ;
- un filtre sert à **modifier une valeur** avant qu'elle soit utilisée.

Exemple du plugin :

```
add_filter('woocommerce_shipping_methods', 'gomile_shipment_add_shipping_method'
);

function gomile_shipment_add_shipping_method($methods) {
    $methods['gomile_shipment'] = 'WC_Gomile_Shipment_Method';
    return $methods;
}
```

WooCommerce fournit la liste des méthodes d'expédition disponibles. Le plugin ajoute son identifiant `gomile_shipment` et retourne la liste modifiée.

3.4 Définition de points d'extension

Un point d'extension défini par un plugin s'appuie sur :

```
do_action('nom_action', $arg1, $arg2);  
$value = apply_filters('nom_filtre', $value, $context);
```

Dans ce plugin, le client API expose deux filtres internes :

```
$headers = apply_filters(  
    'gomile_shipment_delivery_headers',  
    $headers,  
    $context  
);  
  
$request_args = apply_filters(  
    'gomile_shipment_delivery_request_args',  
    $request_args,  
    $url,  
    $context  
);
```

Ces filtres permettent à une autre extension WordPress ou à un mu-plugin (Must-Use Plugin : extension chargée automatiquement depuis `wp-content/mu-plugins`, sans activation manuelle dans l'administration) de modifier les en-têtes ou les arguments HTTP sans modifier directement le code Gomile.

3.5 Fonctions de rappel

WordPress accepte plusieurs formes de fonctions de rappel, appelées *callbacks* dans la documentation technique :

```
add_action('hook', 'ma_fonction');  
add_action('hook', array('Ma_Classe', 'ma_methode_statique'));  
add_action('hook', array($objet, 'ma_methode'));
```

Le plugin utilise principalement des fonctions procédurales et des méthodes statiques :

```
add_action('admin_menu', array(__CLASS__, 'register_menu'));
```

`__CLASS__` vaut le nom de la classe courante, ici `Gomile_Shipment_Admin_Settings`.

3.6 Options WordPress

WordPress fournit une table `wp_options` pour stocker de la configuration. Les fonctions principales sont :

```
get_option('nom_option', $default);  
update_option('nom_option', $value);
```

Le plugin stocke tous ses réglages dans une seule option :

```
const OPTION_NAME = 'gomile_shipment_settings';
```

La classe `Gomile_Shipment_Admin_Settings` expose des méthodes d'accès à la configuration :

- `get_defaults()` : valeurs par défaut ;
- `get_settings()` : valeurs finales après fusion ;
- `get_option($key)` : lecture d'une clé ;
- `has_api_key()` : vérification de présence de la clé API.

Ordre de priorité du plugin :

1. constantes PHP ou variables d'environnement ;
2. option stockée en base ;
3. valeurs par défaut.

3.7 API de réglages WordPress

La Settings API est l'API WordPress utilisée pour déclarer une page de réglages, ses sections et ses champs. Le plugin l'utilise dans `register_settings()` :

```
register_setting(  
    'gomile_shipment_settings_group',  
    self::OPTION_NAME,  
    array(  
        'sanitize_callback' => array(__CLASS__, 'sanitize_settings'),  
        'default'           => self::get_defaults(),  
    )  
);
```

Points importants :

- `sanitize_callback` nettoie toute la configuration avant écriture.

- `settings_fields()` ajoute les champs techniques WordPress, dont la protection du formulaire.
- `do_settings_sections()` affiche les sections et champs déclarés.

3.8 Capacités

WordPress ne vérifie pas seulement si un utilisateur est connecté. Il vérifie des capacités :

```
current_user_can('manage_woocommerce')
current_user_can('manage_options')
```

Le plugin donne priorité à `manage_woocommerce` pour les gestionnaires WooCommerce, puis utilise `manage_options` pour les administrateurs WordPress.

3.9 Nettoyage et échappement des données

En WordPress, deux traitements doivent être distingués :

- **nettoyage** : normaliser une donnée à l'entrée ;
- **échappement** : protéger une donnée au moment de l'affichage.

Exemples utilisés dans le plugin :

```
sanitize_text_field($value);    // texte simple
sanitize_textarea_field($value); // texte multiligne
sanitize_key($value);           // identifiant de statut
esc_url_raw($value);            // URL stocke
esc_html($value);               // texte affich en HTML
esc_attr($value);               // attribut HTML
esc_textarea($value);           // contenu textarea
```

Les données issues d'un formulaire, d'une requête HTTP, d'un webhook, d'une option ou d'un objet WooCommerce sont considérées comme non fiables jusqu'à leur validation explicite.

3.10 Internationalisation

L'internationalisation, souvent abrégée `i18n`, consiste à préparer le code pour qu'il puisse être traduit sans modifier les fichiers PHP. Dans WordPress, cela repose sur le système `gettext`.

Le plugin utilise le text domain `gomile-shipment`. Ce domaine identifie toutes les chaînes traduisibles du plugin et permet à WordPress de charger le bon catalogue de traduction.

Fichiers utilisés

Les fichiers de traduction sont dans `languages/` :

- `gomile-shipment.pot` : template source. Il contient toutes les chaînes détectées dans le code, mais pas les traductions.
- `gomile-shipment-fr_FR.po` : fichier éditable pour la traduction française. Il associe chaque chaîne source à une chaîne traduite.
- `gomile-shipment-fr_FR.mo` : fichier compilé, binaire, chargé par WordPress en production.
- `gomile-shipment.mo` : fichier compilé générique présent dans le projet. En pratique, WordPress charge surtout les fichiers nommés avec la locale, comme `gomile-shipment-fr_FR.mo`.

Fonctionnement au chargement

Le chargement se fait dans le bootstrap du plugin :

```
load_plugin_textdomain(  
    'gomile-shipment',  
    false,  
    dirname(plugin_basename(__FILE__)) . '/languages'  
);
```

WordPress regarde la locale active du site, par exemple `fr_FR`, puis cherche un fichier `.mo` compatible dans le dossier `languages/`. Si le fichier existe, les appels `gettext` retournent la traduction. Sinon, ils retournent la chaîne source.

Fonctions WordPress à utiliser

Toute chaîne affichée à l'utilisateur doit passer par une fonction de traduction avec le domaine `gomile-shipment` :

```
__('Gomile Shipment', 'gomile-shipment');  
esc_html__('Settings', 'gomile-shipment');  
esc_attr__('Copy webhook URL', 'gomile-shipment');  
_x('None', 'Tax status', 'gomile-shipment');
```

Le plugin s'appuie principalement sur les fonctions suivantes :

- `__()` : retourne la traduction sans l'afficher.
- `_e()` : affiche directement la traduction. À éviter si un échappement spécifique est nécessaire.
- `esc_html__()` : traduit puis échappe pour du texte HTML.
- `esc_attr__()` : traduit puis échappe pour un attribut HTML.
- `_x()` : traduit avec un contexte. Utile quand un même mot peut avoir plusieurs sens.
- `_n()` : gère le singulier/pluriel.

Principes appliqués dans le plugin :

- Dans du HTML visible, préférer `esc_html__()`.
- Dans un attribut `title`, `aria-label`, `placeholder` ou `value`, préférer `esc_attr__()`.
- Dans une note de commande ou une erreur construite avec `sprintf()`, utiliser `__()` puis échapper les valeurs dynamiques selon leur usage.
- Ne jamais concaténer des morceaux de phrase traduisibles si l'ordre des mots peut varier selon la langue. Utiliser `sprintf()` avec des placeholders.

Exemple d'utilisation de placeholders :

```
$note = sprintf(
    __('Gomile order created during %1$s. API order ID: %2$s.', 'gomile-shipment'),
    $trigger,
    $delivery_id
);
```

Outils utilisés

Le projet utilise les outils gettext classiques et WP-CLI :

- `wp i18n make-pot` : scanne le code PHP et génère ou met à jour le fichier `.pot`.
- `msgmerge` : fusionne les nouvelles chaînes du `.pot` dans le fichier `.po` existant sans perdre les traductions déjà faites.
- `msgfmt` : compile le fichier `.po` en `.mo`, format utilisé par WordPress à l'exécution.
- `msgattrib` : aide au contrôle qualité, par exemple pour lister les chaînes encore non traduites.

Commandes de maintenance

Régénérer le catalogue source :

```
wp i18n make-pot apps/plugin/woocommerce \
  apps/plugin/woocommerce/languages/gomile-shipment.pot \
  --domain=gomile-shipment \
  --exclude=node_modules,vendor
```

Fusionner les nouvelles chaînes dans la traduction française :

```
msgmerge --update \
  apps/plugin/woocommerce/languages/gomile-shipment-fr_FR.po \
  apps/plugin/woocommerce/languages/gomile-shipment.pot
```

Compiler le fichier chargé par WordPress :

```
msgfmt --check \
  apps/plugin/woocommerce/languages/gomile-shipment-fr_FR.po \
  -o apps/plugin/woocommerce/languages/gomile-shipment-fr_FR.mo
```

Lister les chaînes non traduites :

```
msgattrib --untranslated \
  apps/plugin/woocommerce/languages/gomile-shipment-fr_FR.po
```

3.11 API HTTP WordPress

Pour appeler une API externe, WordPress recommande son HTTP API :

```
$response = wp_remote_request($url, $request_args);
```

Le plugin centralise les appels dans `Gomile_Shipment_Delivery_API::request()`. Cette méthode :

- construit l'URL finale ;
- ajoute les en-têtes JSON et l'en-tête `x-api-key` ;
- encode le body avec `wp_json_encode()` ;
- appelle `wp_remote_request()` ;
- décode la réponse JSON ;
- transforme les erreurs HTTP en `WP_Error`.

3.12 WP_Error

`WP_Error` est l'objet standard WordPress pour représenter une erreur récupérable. Il évite de lancer des exceptions dans le flux WordPress.

Exemple :

```
return new WP_Error(
    'gomile_missing_customer_phone',
    __('The order does not contain a customer phone number.', 'gomile-shipment')
);
```

Le code appelant vérifie ensuite :

```
if (is_wp_error($response)) {
    // Stocker l'erreur, ajouter une note de commande, arrêter le flux.
}
```

3.13 Transients

Les transients sont un cache temporaire WordPress :

```
set_transient($key, $value, 10 * MINUTE_IN_SECONDS);
$value = get_transient($key);
```

Le plugin met en cache les devis de livraison pour éviter de rappeler l'API à chaque recalcul du panier ou du checkout.

3.14 API REST WordPress

WordPress expose des routes sous `/wp-json`. Le plugin déclare :

```
register_rest_route(
    'gomile-shipment/v1',
    '/webhook',
    array(
        'methods'             => WP_REST_Server::CREATABLE,
        'permission_callback' => array(__CLASS__, 'check_webhook_secret'),
        'callback'            => array(__CLASS__, 'handle_webhook'),
    )
);
```

La route finale est :

```
POST /wp-json/gomile-shipment/v1/webhook
```

Le `permission_callback` est obligatoire : il décide si la requête est autorisée. Ici, le plugin vérifie une signature HMAC envoyée dans l'en-tête `X-Gomile-Webhook-Secret`.

4 Notions WooCommerce essentielles

4.1 Méthode d'expédition

Une méthode d'expédition WooCommerce est une classe qui étend `WC_Shipping_Method`. Elle peut être ajoutée à une zone de livraison et proposer un ou plusieurs tarifs.

Dans ce plugin :

```
class WC_Gomile_Shipment_Method extends WC_Shipping_Method
```

L'identifiant technique est :

```
$this->id = 'gomile_shipment';
```

Cet identifiant est utilisé partout pour savoir si une commande a choisi Gomile.

4.2 Initialisation des méthodes d'expédition

WooCommerce charge les méthodes d'expédition via deux hooks :

```
add_action('woocommerce_shipping_init', 'gomile_shipment_shipping_init');
add_filter('woocommerce_shipping_methods', 'gomile_shipment_add_shipping_method'
);
```

Le premier hook déclare la classe. Le second ajoute la classe dans le registre WooCommerce.

4.3 Zones et instances

Dans WooCommerce, une méthode d'expédition est généralement ajoutée à une zone. Chaque ajout crée une **instance** de méthode. C'est pour cela que le constructeur reçoit `$instance_id`.

Le plugin supporte :

```
$this->supports = array(
    'shipping-zones',
    'instance-settings',
    'instance-settings-modal',
);
```

Cela signifie que la méthode peut être ajoutée dans les zones WooCommerce et configurée par instance.

4.4 calculate_shipping

WooCommerce appelle `calculate_shipping($package)` pour obtenir les tarifs d'une méthode.

```
public function calculate_shipping($package = array()) {  
    $quote = gomile_shipment_get_delivery_quote($package, $this);  
    // ...  
    $this->add_rate($rate);  
}
```

`$package` contient les informations suivantes :

- la destination client ;
- les articles du panier ;
- les quantités ;
- les sous-totaux ;
- parfois les coupons appliqués.

Le plugin construit une charge JSON de devis avec l'adresse d'enlèvement, l'adresse client et le poids estimé. Il ajoute ensuite un tarif avec :

```
$this->add_rate(array(  
    'id'      => $this->get_rate_id(),  
    'label'   => $this->title,  
    'cost'    => $cost,  
    'taxes'   => '',  
));
```

4.5 Commandes WooCommerce

Une commande WooCommerce est représentée par `WC_Order`. Les fonctions et méthodes fréquentes sont :

```
$order = wc_get_order($order_id);  
$order->get_id();  
$order->get_items();  
$order->get_shipping_methods();  
$order->get_billing_phone();
```

```

$order->get_address('shipping');
$order->update_meta_data('_key', 'value');
$order->get_meta('_key', true);
$order->save();
$order->add_order_note('...');
$order->update_status('completed', '...');

```

4.6 Métadonnées de commande

Le plugin stocke l'état Gomile dans les métadonnées de commande :

Métadonnée	Usage
<code>_gomile_shipment_status</code>	Statut courant côté Gomile ou état interne (pending, creating, api_error, etc.).
<code>_gomile_shipment_delivery_id</code>	Identifiant de commande/livraison renvoyé par l'API Gomile.
<code>_gomile_shipment_tracking_url</code>	URL de suivi si l'API en renvoie une.
<code>_gomile_shipment_delivery_created</code>	yes ou no, utilisé pour éviter les créations doublon.
<code>_gomile_shipment_last_error</code>	Dernière erreur API lisible depuis l'admin.
<code>_gomile_shipment_last_status_update</code>	Date de dernière mise à jour reçue par webhook.

4.7 Hooks de validation de commande

Le plugin écoute deux flux de checkout :

```

add_action(
    'woocommerce_checkout_order_processed',
    'gomile_shipment_handle_order',
    10,
    1
);

add_action(
    'woocommerce_store_api_checkout_order_processed',
    'gomile_shipment_handle_block_order',
    10,

```

```
1  
);
```

Le premier correspond au checkout classique. Le second correspond au checkout basé sur les blocs WooCommerce et la Store API.

4.8 Statuts de commande

WooCommerce possède ses propres statuts : `pending`, `processing`, `completed`, `cancelled`, etc.

Le plugin ne remplace pas ces statuts par les statuts Gomile. Il garde le statut Gomile dans les métadonnées. Exception actuelle : quand le webhook reçoit `delivery.status_completed`, le plugin marque la commande WooCommerce en `completed`.

5 Architecture du plugin Gomile

5.1 Séquence de chargement

1. WordPress détecte `gomile-shipment.php`.
2. Le plugin définit ses constantes.
3. Le hook `plugins_loaded` appelle `gomile_shipment_init_plugin()`.
4. Le text domain est chargé.
5. `class-admin-settings.php` est chargé même si WooCommerce est absent.
6. Si WooCommerce est absent, le chargement métier s'arrête.
7. Si WooCommerce est présent, les classes d'expédition, d'API, de gestion des commandes et de points d'accès REST sont chargées.
8. `Gomile_Shipment_REST_Endpoints::init()` branche `rest_api_init`.

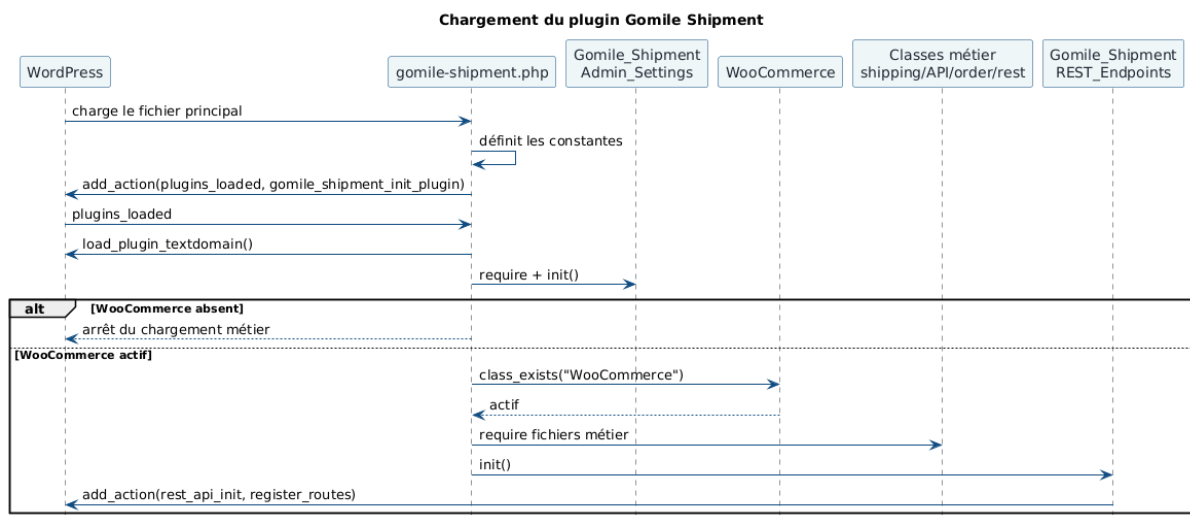


FIGURE 1 – Séquence de chargement du plugin Gomile Shipment.

5.2 Configuration

La configuration est centralisée dans `Gomile_Shipment_Admin_Settings`. Les champs principaux sont :

- URL de base API ;
- clé API ;
- en-tête d'authentification ;
- points d'accès pour le devis, la création, le statut et l'annulation ;
- délai d'expiration HTTP ;
- création automatique de livraison ;
- secret webhook ;
- mode de diagnostic ;
- coordonnées et adresse d'enlèvement.

Les constantes suivantes peuvent surcharger la configuration en base :

```

GOMILE_API_BASE_URL
GOMILE_AUTH_HEADER
GOMILE_AUTH_SCHEME
GOMILE_ENABLE_LIVE_RATES
GOMILE_QUOTE_ENDPOINT
GOMILE_QUOTE_METHOD
GOMILE_QUOTE_CACHE_MINUTES
  
```

GOMILE_CREATE_ENDPOINT
GOMILE_STATUS_ENDPOINT
GOMILE_CANCEL_ENDPOINT
GOMILE_TIMEOUT
GOMILE_AUTO_CREATE
GOMILE_DEBUG_MODE

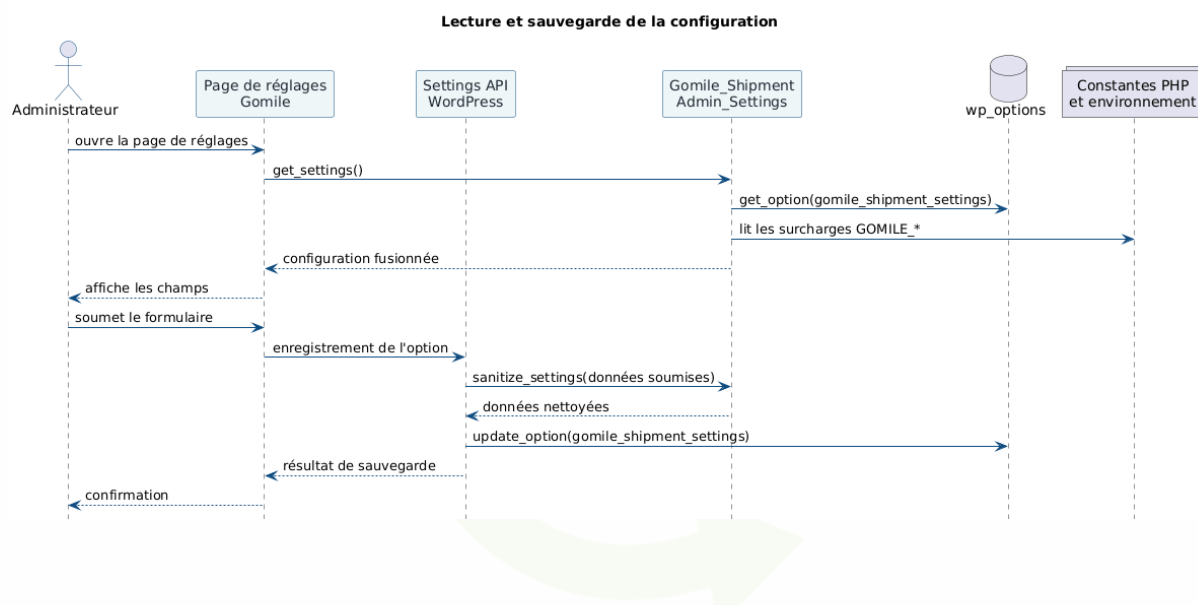


FIGURE 2 – Lecture et sauvegarde de la configuration du plugin.

5.3 Flux de devis

Le flux de devis se produit pendant le panier ou la validation de commande :

1. WooCommerce appelle `WC_Gomile_Shipment_Method::calculate_shipping()`.
2. Le plugin appelle `gomile_shipment_get_delivery_quote()`.
3. `Gomile_Shipment_Delivery_API` construit la charge JSON : adresse d'enlèvement, destination client, poids.
4. Une clé de cache est calculée à partir du package et de la configuration.
5. Si un devis est en cache, il est réutilisé.
6. Sinon, le plugin appelle le point d'accès `/delivery-estimates`.
7. Le champ `deliveryFee` de la réponse devient le coût WooCommerce.
8. Le tarif est ajouté avec `add_rate()`.

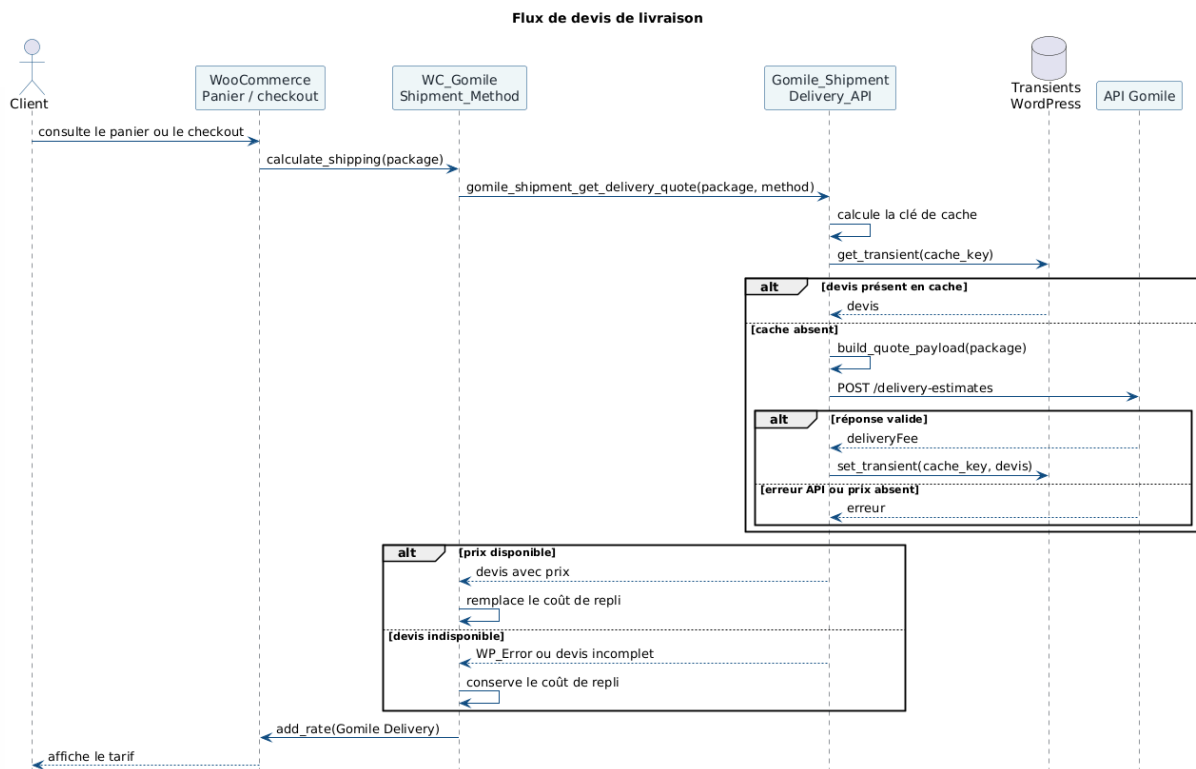


FIGURE 3 – Séquence de calcul du devis de livraison.

Exemple de charge JSON :

```

{
  "pickupAddress": {
    "streetNumber": "12",
    "streetName": "Rue Exemple",
    "postalCode": "38000",
    "city": "Grenoble",
    "country": "France",
    "fullAddress": "12, Rue Exemple, 38000, Grenoble, France"
  },
  "dropoffAddress": {
    "streetName": "Adresse client",
    "postalCode": "38000",
    "city": "Grenoble",
    "country": "FR",
    "fullAddress": "Adresse client, 38000, Grenoble, FR"
  },
  "weightKg": 2.5
}
  
```

```
}
```

5.4 Flux de création de livraison

Après la validation de commande :

1. WooCommerce déclenche un hook de commande.
2. Le plugin récupère la commande avec `wc_get_order()`.
3. Il vérifie que la méthode d'expédition sélectionnée est `gomile_shipment`.
4. Il initialise les métadonnées Gomile.
5. Si la création automatique est désactivée, le flux s'arrête.
6. Si l'API n'est pas configurée, le statut devient `configuration_missing`.
7. Sinon, le statut passe à `creating`.
8. Le plugin appelle le point d'accès `/plugin/orders`.
9. Si l'appel réussit, il stocke l'identifiant Gomile, le statut, l'URL de suivi éventuelle et ajoute une note de commande.
10. Si l'appel échoue, il stocke `api_error` et la dernière erreur.

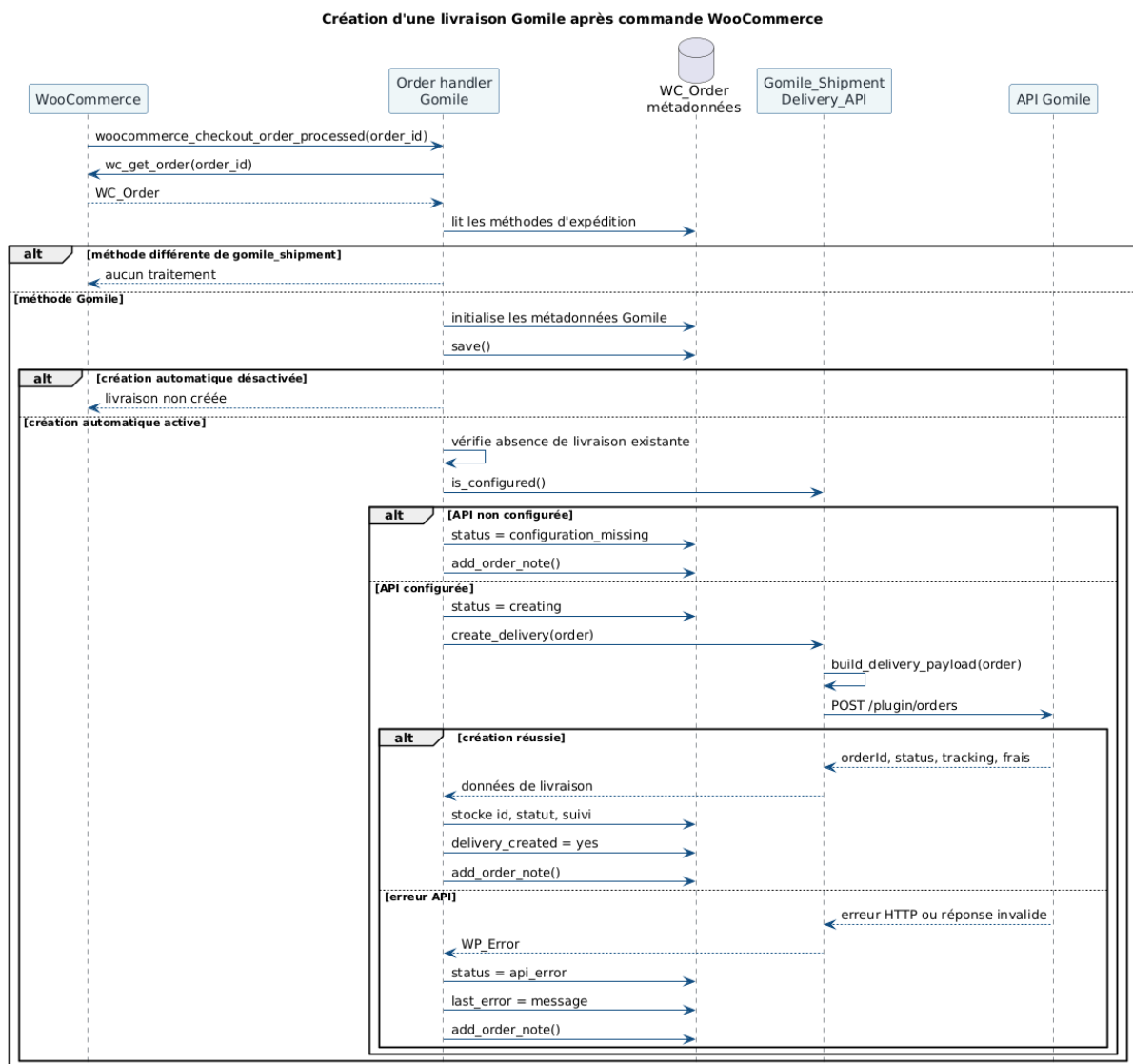


FIGURE 4 – Séquence de création d'une livraison après commande WooCommerce.

Exemple de charge JSON :

```

{
  "customerName": "Marie Martin",
  "customerPhone": "0611223344",
  "dropOffAddress": "12 rue Exemple, 38000, Grenoble, FR",
  "type": "OTHER",
  "weight": 5.0,
  "orderReference": "456"
}
  
```

`orderReference` est l'ID WooCommerce. Le webhook Gomile doit renvoyer la même valeur pour que le plugin retrouve la commande.

5.5 Flux d'annulation

Quand une commande WooCommerce passe en `cancelled` :

```
add_action(
    'woocommerce_order_status_cancelled',
    'gomile_shipment_cancel_delivery_on_order_cancelled',
    10,
    1
);
```

Le plugin vérifie que la commande utilise Gomile, puis appelle :

```
POST /plugin/orders/cancel?orderReference=<id_commande_woocommerce>
```

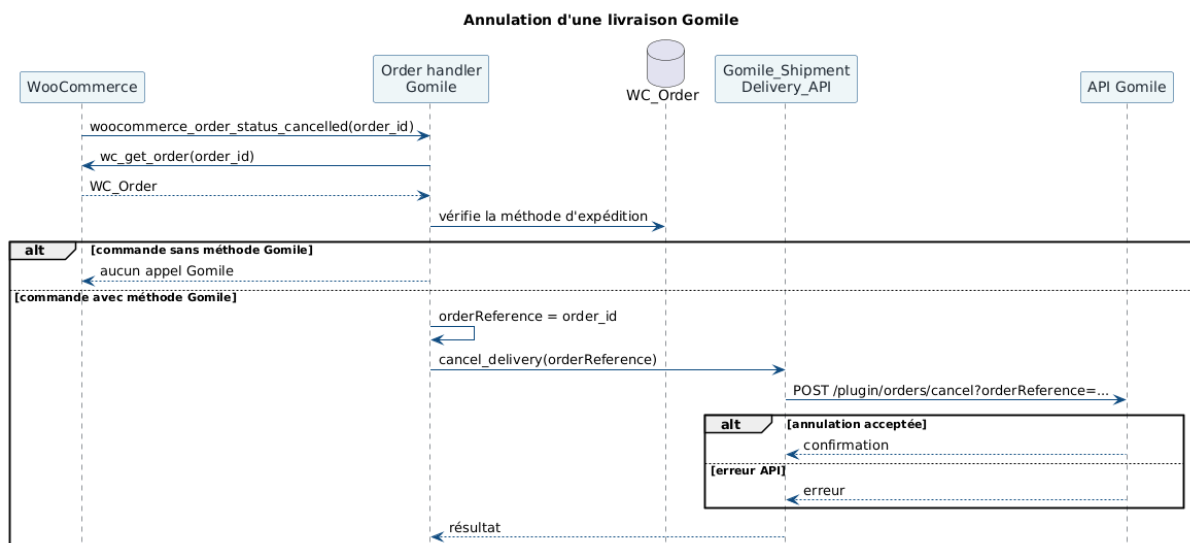


FIGURE 5 – Séquence d'annulation d'une livraison Gomile.

5.6 Flux de webhook

Route WordPress :

```
POST /wp-json/gomile-shipment/v1/webhook
```

Sécurité :

1. le corps brut de la requête est signé côté Gomile avec HMAC-SHA256 ;

2. la signature est envoyée dans `X-Gomile-Webhook-Secret` ;
3. le plugin recalcule la signature avec le secret stocké ;
4. `hash_equals()` compare les deux valeurs.

Format attendu de l'en-tête :

`X-Gomile-Webhook-Secret: sha256=<signature>`

Événements actuellement gérés :

- `delivery.status_changed` : met à jour les métadonnées Gomile et ajoute une note de commande ;
- `delivery.status_completed` : fait la même mise à jour puis marque la commande WooCommerce en `completed`.

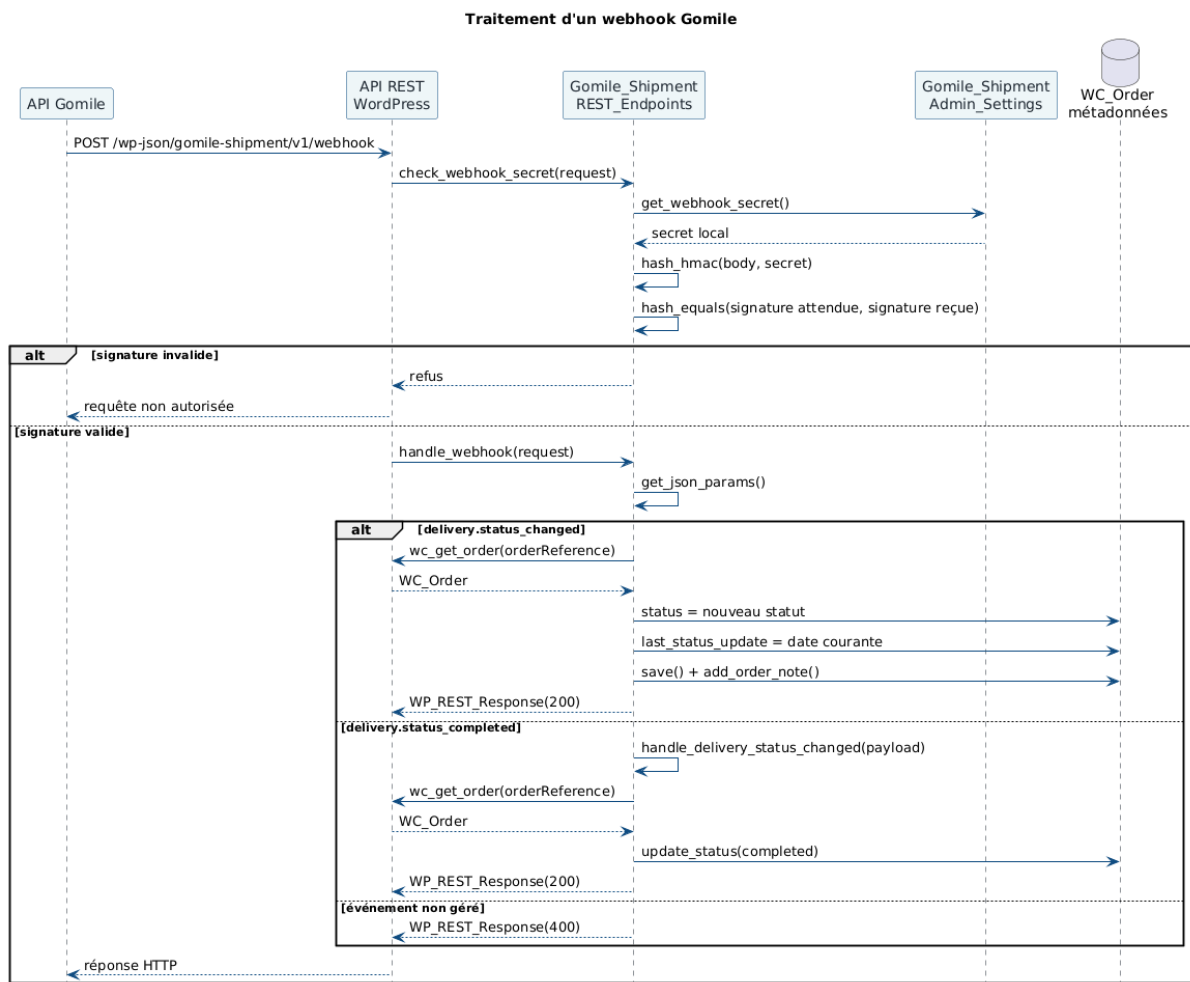


FIGURE 6 – Séquence de traitement d'un webhook Gomile.

6 API Gomile utilisée

Usage	Méthode	Point d'accès par défaut	Code
Devis	POST	/delivery-estimates	get_delivery_quote()
Création commande	POST	/plugin/orders	create_delivery()
Annulation	POST	/plugin/orders/cancel	cancel_delivery()

L'en-tête d'authentification par défaut est :

```
x-api-key: <cle API de la boutique>
```

7 Points d'extension du plugin

7.1 Modifier les en-têtes API

```
add_filter(  
    'gomile_shipment_delivery_headers',  
    function ($headers, $context) {  
        $headers['X-Custom-Header'] = 'value';  
        return $headers;  
    },  
    10,  
    2  
);
```

7.2 Modifier les arguments HTTP

```
add_filter(  
    'gomile_shipment_delivery_request_args',  
    function ($args, $url, $context) {  
        $args['timeout'] = 30;  
        return $args;  
    },  
    10,
```

```
3  
);
```

7.3 Modifier le tarif WooCommerce final

```
add_filter(  
    'gomile_shipment_shipping_rate',  
    function ($rate, $package, $method) {  
        $rate['label'] = 'Livraison locale Gomile';  
        return $rate;  
    },  
    10,  
    3  
);
```

8 Sécurité et robustesse

8.1 Secrets

- La clé API et le secret webhook ne sont pas réaffichés dans les champs de type password.
- Si un champ secret est soumis vide, le plugin garde la valeur existante.
- Les journaux masquent l'en-tête d'authentification configuré.

8.2 Webhooks

Le webhook ne doit jamais accepter une requête sans signature valide. Le plugin refuse la requête si :

- le secret local est vide ;
- l'en-tête de signature est vide ;
- la signature recalculée ne correspond pas.

8.3 Idempotence

La création de livraison évite les doublons avec :

- `_gomile_shipment_delivery_created = yes;`

- présence de `_gomile_shipment_delivery_id`;
- statut temporaire `creating`.

Cette logique limite les créations en double. Les hooks de checkout peuvent être déclenchés dans plusieurs contextes, et les appels réseau peuvent être rejoués.

9 Tests

Les tests unitaires utilisent des doublures WordPress/WooCommerce. Ils ne nécessitent pas une installation WordPress complète.

Test	Comportement vérifié
<code>AdminSettingsTest</code>	Fusion de configuration, conservation des secrets, nettoyage des points d'accès.
<code>DeliveryApiTest</code>	Correspondance entre commande WooCommerce et charge JSON Gomile, devis, annulation avec paramètre <code>orderReference</code> .
<code>OrderHandlerTest</code>	Détection de la méthode d'expédition <code>gomile_shipment</code> sur une commande.
<code>RestEndpointsTest</code>	Signature HMAC, mise à jour de statut, passage au statut <code>completed</code> .

Exécution complète de la suite de tests :

```
cd apps/plugin/woocommerce
composer test
```

Vérification syntaxique et tests unitaires :

```
composer lint
composer test
```

10 Glossaire

Action Hook WordPress qui déclenche un comportement.

Filtre Hook WordPress qui reçoit une valeur, la modifie et la retourne.

Callback Fonction ou méthode appelée par un hook.

Capability Permission WordPress, par exemple `manage_woocommerce`.

Option Valeur de configuration stockée dans `wp_options`.

Transient Cache temporaire WordPress.

Shipping method Méthode de livraison WooCommerce.

Shipping zone Zone géographique WooCommerce dans laquelle une méthode d'expédition est disponible.

Rate Tarif retourné par une méthode de livraison.

Package Structure WooCommerce qui représente le panier à expédier.

WC_Order Objet WooCommerce représentant une commande.

Order meta Métadonnée attachée à une commande WooCommerce.

Webhook Requête HTTP entrante envoyée par Gomile vers WordPress pour notifier un événement.

HMAC Signature cryptographique basée sur un secret partagé.

11 Références officielles

- WordPress `add_action` : https://developer.wordpress.org/reference/functions/add_action/
- WordPress `add_filter` : https://developer.wordpress.org/reference/functions/add_filter/
- WordPress REST API : <https://developer.wordpress.org/rest-api/>
- WooCommerce hooks, actions et filtres : <https://developer.woocommerce.com/docs/extensions/core-concepts/adding-actions-and-filters>
- WooCommerce Shipping Method API : <https://developer.woocommerce.com/docs/features/shipping/shipping-method-api/>
- WooCommerce REST API : <https://developer.woocommerce.com/docs/apis/rest-api/>